Failure Management for Classical Frontier Exploration under a Safety-Filtered Navigation Stack

Author A Author B

Affiliation withheld

Draft for internal review, not a conference submission

Working tree `main` at `de8209a`
Explorer last changed in `eccbb97`

Abstract

This paper describes a frontier explorer that sits above an unmodified Nav2 stack and treats mapping failures as part of the decision, not only as navigation faults. The detector is the classical one: unknown occupancy cells with a free neighbour are clustered, and each cluster is scored by normalised size, Euclidean distance, heading change, and local free-space fraction. Around that score the implementation adds four mechanisms aimed at indoor maps that contain furniture voids, goals the lidar cannot clear, and stops raised by a separate laser safety filter. Enclosed unknown cells are removed by a border-seeded flood fill and then excluded by an exact cell set. A goal that Nav2 reports as reached, while unknown cells remain nearby, is counted as a phantom success and is blacklisted after two such arrivals. A standby queue of spatially separated large frontiers is popped when the safety filter, a progress stall, or a goal timeout fires. Parameters loosen on a fixed schedule if no candidate survives, but the node never latches a terminal "done" state. The repository records why several earlier thresholds were abandoned. It does not record a controlled comparison against nearest-frontier selection or against `explore_lite`. The manuscript therefore states a research question and a method, and it separates that method from performance claims the present evidence cannot support.

Keywords: frontier exploration, occupancy grids, failure recovery, Nav2, safety filter, autonomous mapping.

1 Introduction

A frontier is the boundary between space a range sensor has marked free and space it has not yet observed. Sending a mobile robot to that boundary is one of the oldest practical answers to indoor exploration [1]. The rule is easy to implement on an occupancy grid, and it remains the core of widely used ROS explorers [10]. It is also incomplete. A frontier behind a sofa, a frontier the planner cannot reach while a map is growing, and a frontier the robot is forbidden to approach by a hard safety stop all look, to a pure size-and-distance score, like ordinary goals.

The system studied here was built for a differential-drive robot that maps with SLAM Toolbox and navigates with Nav2. A separate node can zero forward velocity when a narrow laser cone is occupied. The exploration code was changed, over roughly ten weeks, in response to behaviours the authors recorded in commit messages: an openness cutoff that rejected every tight indoor frontier, a

map-border margin that rejected every wall, repeated returns to furniture, and time spent standing still after the safety filter had already stopped the robot. Those changes are real. What the repository does not contain is a trial in which coverage, time, path length, or repeated failures were measured against a declared baseline.

The question the implementation is trying to answer is the following. On a classical frontier detector, does an explicit policy for unobservable voids, uncleared but "reached" goals, and safety-filter preemption reduce repeated commitment to the same map regions, without giving up frontiers that are merely narrow? This paper specifies that policy as it executes on `main`. It does not answer the question.

The contribution is therefore a precise account of an implemented policy, not a claim of superiority. In particular, the paper contributes:

1. a statement of the decision rule, including the enclosed-region test, the phantom-success counter, and the safety-triggered queue;
2. the distinction between that rule and stock frontier utility scoring;
3. a record of design reversals that are supported by commits, including the removal and later partial return of an openness floor;
4. an evidence audit: which statements are properties of the source, and which would require experiments that have not been run.

2 Related work

Yamauchi defined a frontier as the free-space boundary of known space and selected the nearest frontier centroid [1, 2]. Later work replaced pure nearest-frontier choice with utility. Information-gain explorers estimate the expected new map area or the entropy reduction of a candidate viewpoint, often along a planned path [4, 5, 3]. Comparative studies show that the ranking among nearest, greedy, and information-weighted rules depends on the environment, and that a bad cost model can erase the benefit of a better gain model [6].

Detection itself has a literature. Wavefront and fast frontier detectors avoid scanning the whole grid on every cycle by growing only from newly changed free cells [7, 8]. Sampling-based detectors, notably multiple rapidly exploring random trees, propose candidates without a full boundary extraction [9]. The implementation in this repository does none of those. Every selection walks the full grid.

The closest deployed relative is `explore_lite`, whose usual citation is Hörner's map-merging thesis [10]. It clusters frontiers, scores them with a weighted gain and a path cost, and can blacklist failed goals. The scorer here is in that family, with two differences that matter. Cost is Euclidean distance in the map frame, not the length of a Nav2 plan, so a frontier behind a wall can outrank a longer but open route. Gain is the cell count of the unknown cluster, not a ray-cast estimate of newly visible cells.

Nav2 and SLAM Toolbox are used as unmodified libraries [11, 12]. The local controller is regulated pure pursuit [13]. Nothing in the explorer replaces the planner, the costmap, or the behaviour tree. No custom behaviour tree is checked into the repository, so recovery order inside Nav2 is the distribution default for the configured plugins `spin`, `backup`, and `wait`.

What is not claimed as new is frontier clustering, a linear utility, a blacklist, or flood fill of unreachable cells. Hole filling and reachable-space tests are standard mapping operations. The

specific packaging that this code adds, and that a reviewer should judge only after a prior-art search, is the joint use of (i) a permanent exact-cell exclusion of enclosed unknown space, (ii) a counter that fires on Nav2 *success* when the frontier remains, and (iii) preemption from an external safety-stop Boolean into a geographically separated queue. Section 7 explains why that packaging is not yet a result.

3 Problem and scope

Let the map be a grid M of width W and height H at resolution ρ. Each cell is treated as free if its value is 0 and unknown if its value is -1. Every other value blocks traversal in the explorer's own tests. This is stricter than a probabilistic reading: a cell at occupancy 50 is neither a frontier seed nor free space.

The robot pose (x, y, ψ) is the `map`→`base_footprint` transform. The explorer's only actuation is a `NavigateToPose` goal. It does not command velocity, and it does not read the camera or the semantic detector. Those nodes run in parallel and do not enter the score.

A frontier cluster F is a connected set of unknown cells, each 8-connected to at least one free cell. The goal point $g(F)$ is not the centroid. It is the free cell, among the 8-neighbours of F, closest in squared index distance to the rounded mean index of F. The robot is therefore sent into known free space beside the boundary. The planner may still cross unknown space, because NavFn is configured with `allow_unknown: true` and a 0.5 m tolerance.

The policy must choose g or declare that it has no candidate, while avoiding three failure modes that the commit history names:

1. goals inside or on unobservable voids, such as the interior of furniture;
2. goals Nav2 can reach in the sense of its goal checker while the unknown cells remain;
3. long waits after the safety filter has already forbidden forward motion.

The policy is not required to be optimal, to maximise information gain, or to explore a known closed world in bounded time. Termination, as implemented, is a log line rather than a stop.

4 System context

Figure 1 shows the nodes that affect exploration. SLAM Toolbox publishes `/map`. The explorer reads that grid, the pose, `/safety_blocked`, and optional manual no-go zones. It sends goals to Nav2. The command path on this Jazzy configuration is controller, velocity smoother, collision monitor, then the custom safety stop, then the motor driver (Fig. 2). Launch comments that omit the smoother and the collision monitor are stale relative to `config/nav2_params.yaml`.

The safety stop uses a $\pm 25^\circ$ forward cone (Fig. 3). A return between 0.30 m and 0.35 m engages the block; release requires 0.45 m, so the threshold does not chatter. Rear motion uses the same angular width. Yaw rate is never blocked, which is what lets Nav2 spin while translation is held. The Boolean published to the explorer is true if the forward cone is blocked or if raw wheel odometry has been silent for 0.5 s. A rear-only obstacle does not preempt exploration. A dead encoder does.

Bringup starts SLAM at 5 s and Nav2 at 8 s. The explorer starts at 22 s. Its decision timer is 3 s, taken from `goal_interval_sec`. All timeouts below are checked on that timer, so a "3 s" safety switch actually occurs on the first tick after the threshold, typically in $(3, 6]$ s.

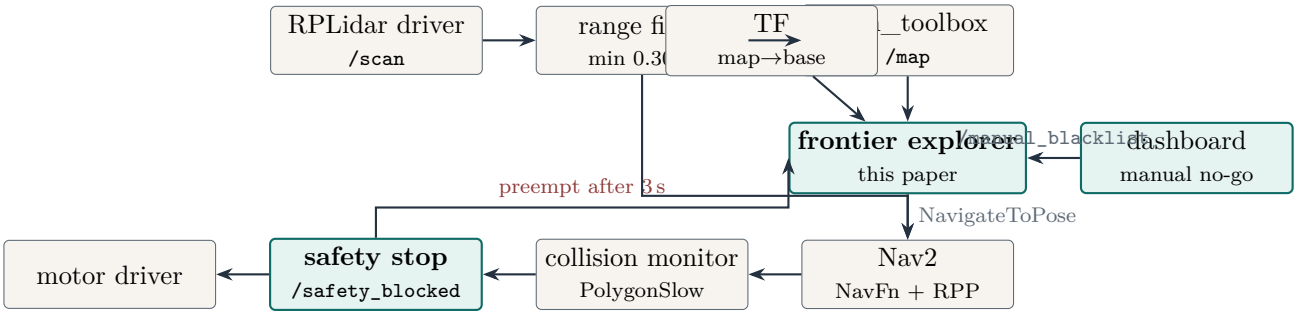


Figure 1: Nodes that participate in exploration. Teal marks code in this repository. The camera and detector are omitted because they do not affect frontier scores.

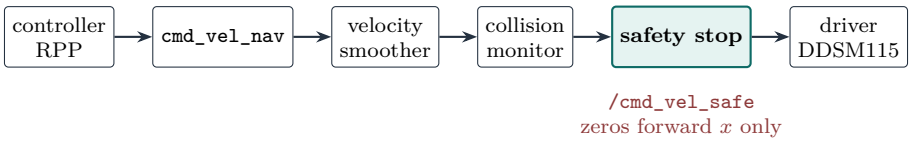


Figure 2: Velocity path implied by the Nav2 parameter file. PolygonSlow can scale a command to half speed before the safety stop independently zeros forward motion.

5 Method

5.1 Detection and enclosed regions

On each full scan the grid is copied into an array. Let U be the unknown cells and C the free cells. Traversable cells are $C \cup U$. A 4-connected flood fill is seeded from every traversable cell on the border of the array. Unknown cells that the fill does not reach are enclosed. They are written as occupied (100) in the copy only. The SLAM map is not edited.

Figure 4 is a schematic of the intended case: a void with no 4-connected path to the map border never becomes a frontier. Frontiers themselves are then extracted with 8-connectivity. A diagonal gap can therefore be "enclosed" for the flood fill and still be a frontier adjacency. That mismatch is a property of the code, not an interpretation of it.

Each enclosed component of at least three cells is stored twice. The centroid is kept for a marker. The cells, expanded by one cell of Chebyshev buffer, are stored as rounded world-coordinate keys $\mathrm{round}(p_x/\rho)$ and $\mathrm{round}(p_y/\rho)$. Later rejection tests are set membership, not a disk. Components whose centroids lie within 1.6 m of an existing permanent centroid are not entered again. The markers still draw a disk of radius 1.6 m, which is larger than the cell set the predicate uses.

The same rounding aliases neighbouring cells. With $\rho = 0.05$ m and an origin on a cell boundary, Python's half-to-even `round` maps cell columns 0 and 1 to different keys but maps columns 1 and 2 onto one key. The permanent mask is therefore not a perfect rendering of the buffered component.

5.2 Pruning

Clusters then pass the cascade in Fig. 5. The minimum size starts at 3 cells. The border test is an index margin $m = \max(1, \mathrm{int}(0.15/\rho))$. At the configured resolution $\rho = 0.05$ m, $0.15/0.05$ evaluates to a float slightly below 3, and `int` truncates it to 2. The comment in the YAML file, and the June reference note, both say three cells. The running code rejects two cells of index margin, about 0.10 m to the cell origin, and it can accept a goal whose centre is 0.125 m from the map border. The test is

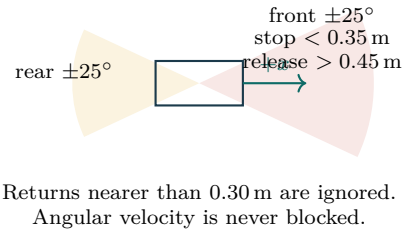


Figure 3: Safety geometry as launched on the hardware bringup. The shaded front sector is the only laser condition that sets `/safety_blocked` for the explorer, together with stale wheel odometry.

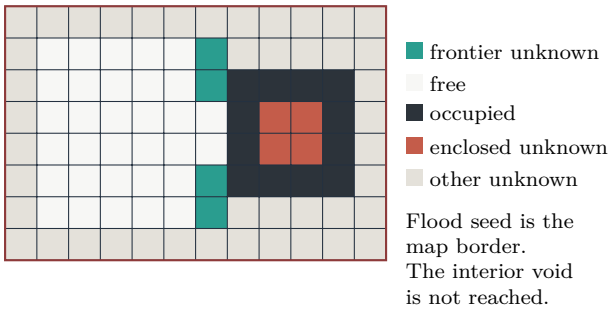


Figure 4: Conceptual grid, not a logged map. The red cells have no 4-connected free or unknown path to the border, so they are removed before clustering. Teal cells are unknown cells that touch free space and can form a frontier.

not a Euclidean distance to the boundary.

Distance limits are $d_{\min} = 0.35\,\text{m}$ and $d_{\max} = 5.5\,\text{m}$ on a full score. If nothing survives, the same clusters are scored again with $d_{\max}$ replaced by $\min(2\,d_{\max},\, 12\,\text{m})$. Openness $\Omega(g)$ is the fraction of cells in a 7×7 window, radius 3 cells, that are free. Candidates with $\Omega < 0.15$ are dropped. A floor of 0.25 was removed on 13 May 2026 because, the commit message says, it rejected all tight frontiers in warehouse and office worlds. The 0.15 floor was reintroduced on 3 June as a narrower wedge filter. Both statements are commit messages, not measured rejection counts.

5.3 Score

For a surviving cluster, with S the largest cluster size in the current list and D the distance limit used for this pass,

$$\begin{aligned} \sigma_{\text{size}} &= |F|/S, & \sigma_{\text{dist}} &= \|g - p\|/D, \\ \sigma_{\text{head}} &= |\text{wrap}(\text{atan2}(g - p) - \psi)|/\pi, & \sigma_{\text{open}} &= \Omega(g). \end{aligned} \tag{1}$$

The utility is

$$J(F) = 1.8\,\sigma_{\text{size}} - 1.0\,\sigma_{\text{dist}} - 0.5\,\sigma_{\text{head}} + 0.7\,\sigma_{\text{open}}. \tag{2}$$

Weights are the YAML values, which override the different defaults compiled into the node. Figure 6 decomposes one hypothetical candidate. It is an arithmetic check of (2), not an observation of the robot.

If every survivor is farther than $0.35D$, the distance term is recomputed as $\max(0,\, \|g-p\|/D - \beta)$ with $\beta = \min(0.40,\, 0.45\,|F|/S)$. A large far cluster sheds more of its distance penalty than a small one. The bonus does not run when any near candidate exists, even if that near candidate later loses on the other terms.

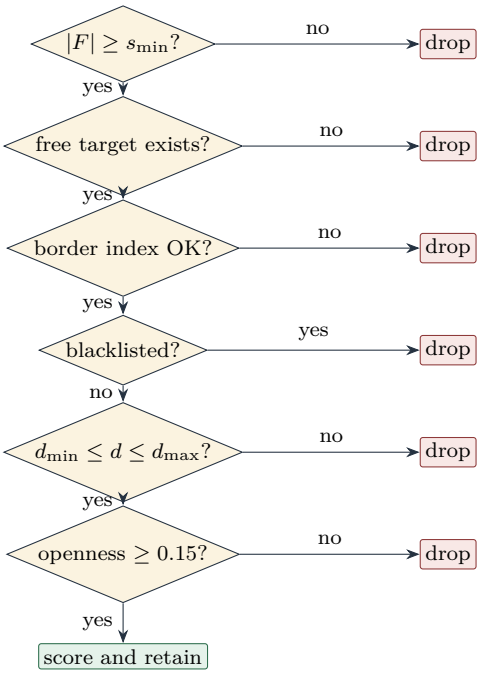


Figure 5: Order of rejection inside scoring. Openness is both a hard floor and a term in the utility.

There is a second score, used only when a queued point is popped:

$$J_{\text{pop}} = -\,\|g - p\|/D_{\text{cfg}}, \tag{3}$$

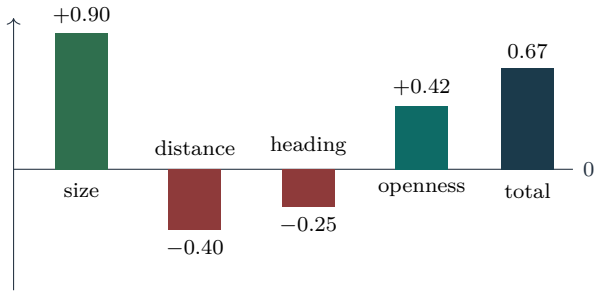
where D_{cfg} is the current search radius, not the cap. The four-term value that ordered the queue is discarded. Figure 7 shows the consequence for the hysteresis test. While a goal is in progress the test is not consulted. On the idle path it is consulted whenever a previous goal position is still stored, and it demands $J \geq J_{\text{active}} + 0.20$. A popped candidate at -0.40 does not beat a stored four-term score of 0.67. Whether that branch stalls a run depends on how long the previous goal position remains set; the result callback usually clears it. The scales are nonetheless not comparable, and the logged "score" on a queued dispatch is not (2).

5.4 Queue, preemption, and blacklist

The highest-scoring survivor is the value returned by a full scan. It is not inserted into the queue. Up to five further survivors are inserted if they contain at least n_L cells and are at least 2.0 m from other queued points, where n_L is fixed at the first map as

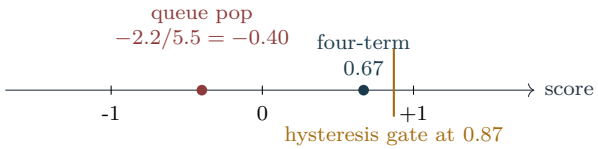
$$n_L = \max(3s_{\min},\, 15,\, \lfloor 0.0015\, WH \rfloor). \tag{4}$$

If no cluster meets n_L, the size rule is dropped and any survivor may fill the queue. Figure 8 shows the geometry. The idle loop pops this queue before it scans again, provided the queue is younger than 20 s. A popped point is rejected if it is blacklisted, if no unknown cell remains within 0.8 m, if it is closer than $d_{\min}$ or farther than the 12 m cap, or if it lies within 1.6 m of the last blacklisted point. The upper bound is the cap, not $d_{\max}$, so a queue can dispatch a goal outside the normal search radius. Because the queue is preferred whenever it is non-empty, a fast success can send a stale runner-up instead of recomputing the current best.



Illustrative inputs, not a trial: $|F| = 40$, $|F|_{\max} = 80$, $d = 2.2\,\text{m}$, $D = 5.5\,\text{m}$, $|\Delta\psi| = \pi/2$, $\Omega = 0.60$.

Figure 6: Worked decomposition of (2). The inputs are chosen for readability. They were not taken from a log.



Same candidate can be logged as 0.67 when it is scored
and as −0.40 when it is later popped. The gate compares them.

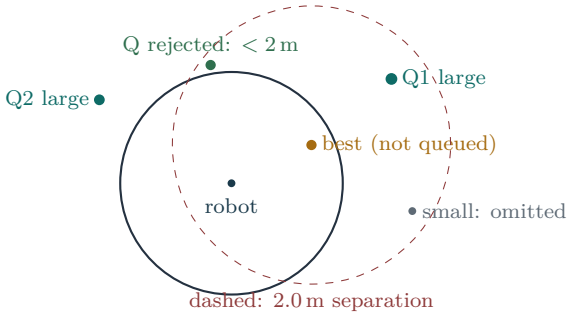
Figure 7: The idle hysteresis gate assumes one score definition. Queue dispatch uses another. The numerical example matches Fig. 6 and (3).

Figure 9 is the goal state machine. Figure 10 lists the transitions that cancel a goal. A vanished frontier is a clean cancel: the area is not blacklisted. A safety preemption, a stall, and a 90 s timeout blacklist first and then cancel. The cancel-status callback does not blacklist again. That double write was a defect fixed on 10 June (`47a164d`). Stalls are defined on Nav2 feedback: the timer resets only when `distance_remaining` falls by more than 0.12 m. A longer replan does not count as progress. Rotation in place can satisfy Nav2's pose progress checker, which accepts 0.3 rad, while the explorer still sees a stall. The two clocks are both set to 25 s. The YAML comment says the explorer timeout must exceed Nav2's allowance. It does not. It equals it, and the quantities differ.

A blacklist entry stores a timestamp, a hit count, and a timeout. Entries within 1.2 m are merged. The first hit uses radius 0.80 m and timeout 300 s. Later hits use radius 1.60 m and timeout $\min(300 \cdot 2^{h-1}, 2400)$ s. A second Nav2 abort at the same stored point additionally forces the timeout to 600 s and the hit counter to 10. The radius does not grow past double; the forced counter does not enlarge it further. The first abort is deliberately not a long exclusion. The commit message states the reason: a costmap resize can abort a goal that is in fact reachable.

Phantom success is separate. On `STATUS_SUCCEEDED`, if any unknown cell remains inside 0.8 m, a counter for a nearby key is incremented. At 2, the point is blacklisted with a temporarily substituted base timeout of 600 s, and the hit counter is forced upward so the radius doubles. The original threshold in `6b7e6c2` was 3. The 3 June report (`29348bf`) still says 3. The threshold became 2 on 10 June (`b79ad0d`). The report is not a description of the present source.

Manual zones arrive as JSON on `/manual_blacklist`. The dashboard converts a click from `odom` to `map`. Duration ≤ 0 does not expire. These zones are ordinary disks and are independent of the permanent cell set.



Conceptual geometry. The best cluster is returned as the immediate goal and is not stored in the queue. The idle loop later pops the queue before it rescans.

Figure 8: How the standby set is built. Separation uses the blacklist radius times 2.5, which is 2.0 m at the configured 0.80 m radius. The circle is illustrative.

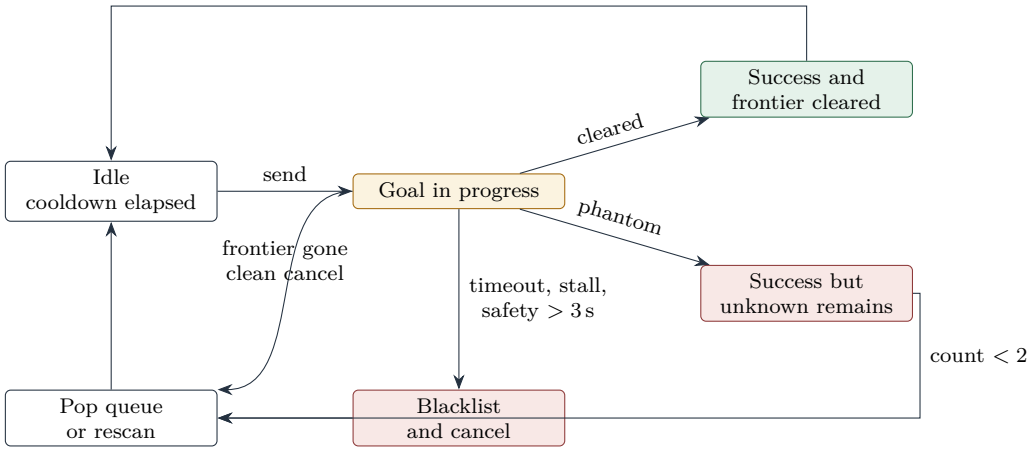


Figure 9: Goal lifecycle implemented by the timer and the action callbacks. "Complete" is not a state.

5.5 Calibration, relaxation, and stopping

The first occupancy grid, and only the first, may raise $d_{\max}$ to $\min(0.5L,\ 12\,\text{m})$ if that value exceeds 5.5 m, where L is the larger side of the grid in metres. It may raise the empty-tick patience to $\max(20,\ \lfloor 2L \rfloor)$. It sets n_L by (4). This uses the SLAM canvas, which includes unknown margin, and it never repeats. A first message that is smaller than the eventual map leaves the search radius at 5.5 m. A first message that is a large empty canvas inflates n_L. On a 400×400 grid, $n_L = 240$ cells, so the "large frontier" queue is often empty and the fallback admits small clusters.

If a tick finds nothing, constraints loosen and stay loose (Fig. 11). From the configured size of 3: tick 3 sets the minimum size to 2; tick 6 sets it to 1 and extends the search radius to 12 m; tick 9 sets $d_{\min}$ to 0.175 m. At tick 20 the node logs that exploration is complete. It does not set a flag, cancel the timer, or refuse later goals. The June report says the node stops sending goals. The source does not.

5.6 What the first map does to later decisions

Figure 12 shows the only place the map size enters the policy. It does not estimate the explored area. It copies the dimensions of whichever grid arrives first into three knobs that then shape scoring,

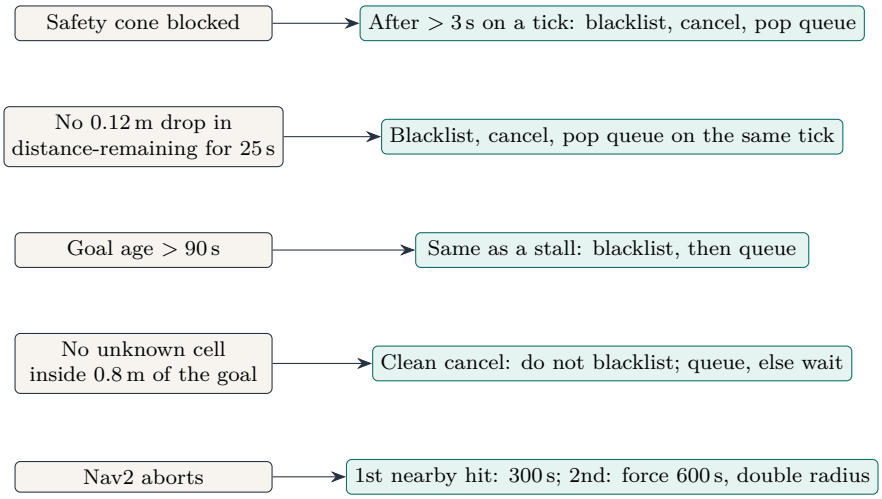


Figure 10: Which events blacklist the active goal. Durations are evaluated on the 3 s timer, so the effective delay is the threshold plus up to one period.

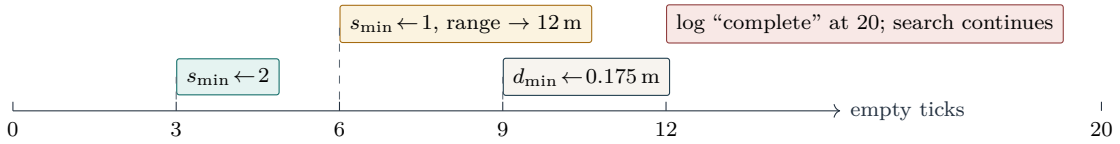


Figure 11: Sticky relaxation for the configured initial size of three cells. The schedule is in ticks of the 3 s timer, and only ticks that find no candidate increment the counter.

queue admission, and the meaningless completion log.

6 What was tried before

Table 1 keeps only changes that alter exploration behaviour. Hardware, teleoperation, and the cloud bridge are omitted. Motivations in the middle column are quoted from commit messages or from comments added in those commits. Where the message is silent, the cell says so.

A home-return behaviour exists on branch `home_updated` and is not on `main`. After six empty ticks that branch sends the robot to the pose captured at start. No commit on `main` explains the omission. It should not be described as part of the method.

7 Evidence audit

No rosbag, CSV, coverage trace, or repeat-trial table is in the repository. Commit messages in July 2026 say that particular navigation faults were checked on the robot: a transform tolerance of 1.0 s after map-to-odometry lag near 0.6 s, a narrower safety cone after sideways obstacles froze the robot, and a halt when wheel odometry stopped. Those notes are not exploration experiments. They do not report sample size, a baseline, or an outcome distribution. This paper does not convert them into performance numbers.

The following statements are true because they are source behaviour, and only for that reason.

- The detector is a full-grid classical frontier extractor with a linear utility.

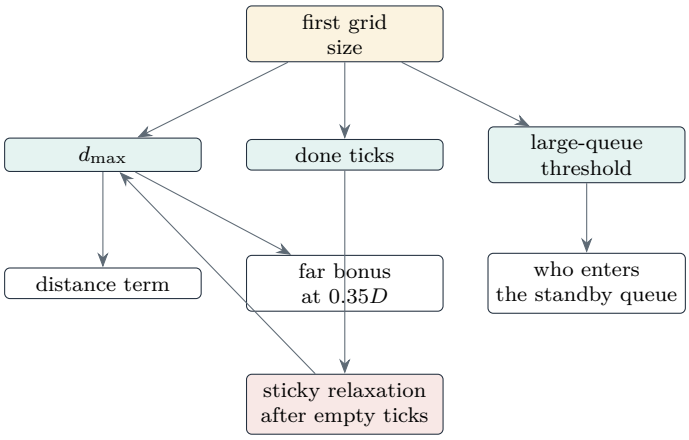


Figure 12: Parameter interactions that are computed, not tuned online. Relaxation can later overwrite the calibrated search radius. It does not recompute the queue threshold.

- Enclosed unknown cells are excluded from detection for the rest of the process, and their cells are an exact blacklist.
- Nav2 success with residual unknown is counted, and the second count blacklists.
- The safety Boolean can preempt a goal without waiting for the 25 s stall timer.
- The completion log does not stop the node.
- The June 2026 reference report disagrees with the current source on the phantom threshold, the shape of permanent zones, the safety cone, and termination.

The following statements are not supported.

- The policy explores faster, shorter, or more completely than nearest-frontier or `explore_lite`.
- The far-frontier bonus, the queue, or the phantom counter is responsible for any share of an improvement.
- Two phantom arrivals are a better threshold than three.
- The 0.15 openness floor removes only unreachable goals.
- Exploration terminates.

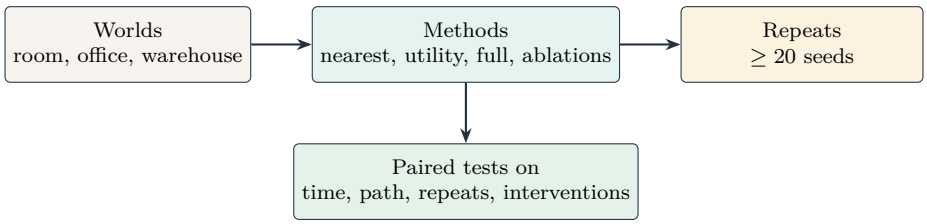
Figure 13 is the minimum experiment that would start to support a conference claim. It has not been executed. Worlds already named in the history, office and warehouse, are the natural simulated maps; a hardware floor plan should be added only with a logged map. Each method should share the same Nav2 and SLAM configuration. Ablations are: no enclosed fill, no phantom counter, no queue, no far bonus, and no relaxation. Outcomes are time to a coverage threshold, path length, number of goals, number of goals within 1.2 m of a previous failure, safety-preemption count, and planner aborts. At least twenty repeats and a paired comparison against nearest-frontier and a gain-over-cost utility are required before any ranking is stated. Until those data exist, charts of exploration performance would be fiction, and this paper does not contain them.

8 Threats to validity and limitations

Even after those trials, several structural choices limit what a positive result would mean.

Table 1: Algorithmic revisions that survive in, or explain, the current policy. Hashes are the introducing commits.

Date	Change	Stated reason
12 May	Baseline utility, expansion, openness floor 0.25, border 0.60 m	Restored after a branch deletion (`f493c96`).
13 May	Openness floor removed	"Rejecting all tight-space frontiers" (`82dbf87`).
13 May	One-shot calibration and relaxation	Commit title only (`48d893c`). No trial log.
24 May	Border 0.60 → 0.15 m; blacklist 0.50 m/90 s → 0.80 m/300 s	YAML comment: 0.60 m removed every wall-adjacent frontier.
2 Jun	Queue, safety preemption, vanish-cancel, repeat radius	"Zero standstill time" is the commit's goal, not a measurement (`827026d`).
3 Jun	Phantom counter, enclosed fill, far bonus, two-stage abort	Messages name occlusions, furniture voids, and costmap-resize aborts.
10 Jun	Threshold 3 → 2; exact-cell permanent zones; callback generation	Replaces a 1.6 m permanent disk (`5bbaf0f`).
19 Jul	Manual no-go	Dashboard zones (`eccbb97`).



Not yet run. No chart in this paper uses measured exploration outcomes.

Figure 13: Evaluation design. The figure is a protocol, not a result.

1. Euclidean cost ignores walls. A win over nearest-frontier might disappear once both methods use path length, which is the cost already available from Nav2.
2. Cluster size is not visibility. A long frontier seen at a grazing angle scores like a frontier that would reveal a room.
3. The 3 s timer quantises every preemption. A safety stop just after a tick waits almost another full period.
4. One-shot calibration on the SLAM canvas couples a middleware detail to the decision thresholds.
5. Permanent cell keys alias under `round`, and the RViz disk overstates the exclusion.
6. The pop-score in (3) makes hysteresis and logs inconsistent with (2).
7. A dead wheel encoder is reported on the same Boolean as a forward obstacle, so a driver fault blacklists a goal that may have been feasible.
8. Semantic perception is unused. The method cannot prefer a doorway it has seen, and it also

cannot be credited with semantic exploration.

Computational cost is unprofiled. Each full scan is a linear pass over WH for masks and flood fills, plus an 8-connected clustering of the frontier cells and a 7×7 window per survivor. At the 3 s idle period this is unlikely to dominate a Raspberry Pi that is already scan-matching, but that sentence is a complexity observation, not a timing measurement. During a goal, a scan is repeated only when the queue is older than 20 s.

9 Conclusion

The repository contains a classical frontier explorer wrapped in a failure policy: drop enclosed unknown space by an exact cell set, treat an uncleared success as different from a planner abort, and leave a safety stop by a spatially separated standby goal. The policy is specific enough to reimplement from this paper together with the parameter table in the companion technical report. It is not specific enough, and not tested enough, to support a claim at IROS, ICRA, or RA-L that the robot explores better.

The defensible next paper is short. Fix the score so that a queued goal and a freshly scored goal are the same function. Latch termination or delete the claim. Log the counters the audit asks for. Run the protocol in Fig. 13 against nearest-frontier and a path-cost utility, with the enclosed-region test and the phantom counter ablated. If those two mechanisms move repeated failures and do not move coverage the wrong way, there is a result. If they do not, the honest finding is that the engineering did not change the outcome, which is also publishable and which the present documents cannot yet show.

A public Apache-2.0 repository already discloses the method. Questions of patentability are treated in the separate assessment. They should be answered by counsel before a conference version of this text is posted, and they should not be answered by treating the implementation as novel because it is local.

References

[1] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.

[2] B. Yamauchi, "Frontier-based exploration using multiple robots," in *Proc. Int. Conf. Autonomous Agents*, 1998, pp. 47–53.

[3] H. H. González-Baños and J.-C. Latombe, "Navigation strategies for exploring indoor environments," *Int. J. Robotics Research*, vol. 21, no. 10–11, pp. 829–848, 2002.

[4] A. A. Makarenko, S. B. Williams, F. Bourgault, and H. F. Durrant-Whyte, "An experiment in integrated exploration," in *Proc. IEEE/RSJ IROS*, 2002.

[5] C. Stachniss, G. Grisetti, and W. Burgard, "Information gain-based exploration using Rao-Blackwellized particle filters," in *Proc. Robotics: Science and Systems*, 2005.

[6] D. Holz, N. Basilico, F. Amigoni, and S. Behnke, "Evaluating the efficiency of frontier-based exploration strategies," in *Proc. ISR/ROBOTIK*, 2010.

[7] M. Keidar and G. A. Kaminka, "Efficient frontier detection for robot exploration," *Int. J. Robotics Research*, vol. 33, no. 2, pp. 215–236, 2014.

[8] P. Quin, A. Alempijevic, G. Paul, and D. Liu, "Expanding wavefront frontier detection: an approach for efficiently detecting frontier cells," in *Proc. Australasian Conf. Robotics and Automation*, 2014.

[9] H. Umari and S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," in *Proc. IEEE/RSJ IROS*, 2017.

[10] J. Hörner, "Map-merging for multi-robot system," Bachelor's thesis, Charles University, Prague, 2016. Implementation basis of `explore_lite`.

[11] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.

[12] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *J. Open Source Software*, vol. 6, no. 61, art. 2783, 2021.

[13] S. Macenski, S. Singh, F. Martín, and J. Ginés, "Regulated pure pursuit for robot path tracking," *Autonomous Robots*, vol. 47, pp. 685–694, 2023.